典型应用案例

围绕术语知识管理、垂直文化场景与大模型交互，形成三个可复用的应用入口

01 CASE 01 通用知识管理

多语术语检索

面对分散的中华文化术语，用户需要在同一入口快速定位可靠译法与文化释义。

核心链路

关键词/筛选 → 分页查询 → 结构化详情

能力要点

- 模糊搜索与文化、场景、层级组合筛选
- 中英及小语种官方译法统一呈现
- 儿童模式切换为更易理解的解释

业务价值

降低术语查找与理解成本

02 CASE 02 垂直领域

淮河文化与中药材

文化传播与中药材跨语种使用中，译名、编码和合规信息需要关联呈现。

核心链路

专题搜索 → 多语字段聚合 → 风险/场景提示

能力要点

- 淮河术语、人物与中药材专题数据
- 拉丁名、HS 编码与 8 类多语译名字段
- 合规提示、翻译风险和适用场景

业务价值

支持专业术语的跨语种复用

03 CASE 03 AI 应用接入

AI 文化问答

面对陌生文化概念，用户可直接提问，并连续获得通俗、简洁的解释。

核心链路

用户问题 → Flask AI 代理 → SSE 增量回答

能力要点

- 系统角色限定为中华文化术语助手
- 模型配置通过环境变量与业务代码分离
- 流式数据块实时追加并以 [DONE] 结束

业务价值

缩短从检索到理解的路径

统一价值链

用户需求 → 术语检索 / 专业场景 / AI 问答 → 结构化结果与连续交互

系统架构图

React 单页应用通过 REST 与 SSE 访问 Flask 服务，业务数据、离线导入与外部模型调用保持清晰边界

CURRENT / 当前已实现



核心流程与 AI 演进

左、中两列为当前实现；右列以虚线展示后续可演进的 RAG 知识增强路径

A 术语查询流程

搜索 / 筛选 / 分页 / 详情

1. **输入查询条件**
   关键词或文化 / 场景 / 层级
2. **前端组装参数**
   React 状态映射为查询参数
3. **调用 REST 接口**
   /api/search 或 /api/filter
4. **校验并计算分页**
   page、pageSize 与 offset
5. **执行 ORM 查询**
   select、count、like 与组合条件
6. **返回并渲染**
   统一 JSON → 列表 / 详情弹窗

B AI 流式问答

豆包 API + SSE 增量输出

1. **提交用户问题**
   POST /api/chat/stream
2. **接口前置校验**
   空消息 400；未配置模型 503
3. **调用豆包模型**
   chat.completions，stream=True
4. **封装 SSE 数据块**
   data: {"content": "..."}
5. **前端增量解码**
   ReadableStream 持续读取并追加
6. **结束或异常**
   [DONE] / error 数据块

C RAG 知识增强

检索术语知识后再生成回答

演进方向

1. **解析用户问题**
   识别术语、领域与回答意图
2. **检索术语知识**
   关键词 + 向量混合召回
3. **组装可信上下文**
   译法、来源、释义与合规字段
4. **约束模型生成**
   仅基于召回内容组织回答
5. **返回答案与来源**
   支持依据追溯与结果校验
6. **反馈闭环**
   记录未命中问题并补充知识库

接口边界

pageSize 最大 100 | 空消息返回 400 | AI 未配置返回 503 | 异常写入日志并返回 JSON / SSE error